NEOS TIMELINE

From starter template to structured OS shell

This PDF is the public-facing process log for the NEOS rebuild. It is written to be readable by both technical and non-technical reviewers. Each milestone explains what changed, why it changed, and what the UI looked like at that step.

Current coverage: Stage 1A (original shell UI) → SHELL-01A v2 (desktop shell rewrite).

Project Snapshot

The team reset implementation discipline before the v2 rewrite. Work is sequenced by lane and controlled by spec authority order.

- Active implementation lanes: SHELL-DESKTOP, WINDOWING-OSAPI.
- HOME-INTEGRATION is allowed only when it directly supports shell rules.
- CONNECT-SPEC is discussion-only.
- CONNECT-IMPLEMENTATION is currently frozen.

Timeline Map

1. Stage 0: Foundations
2. Stage 1A: First shell UI pass (original HOME channel navigation)
3. SHELL-01A v2: Desktop shell frame rewrite (current)
4. Stage 1B: Runtime bootstrap and core services
5. Stage 2: Window manager behavior
6. Stage 3+: App runtime and core apps

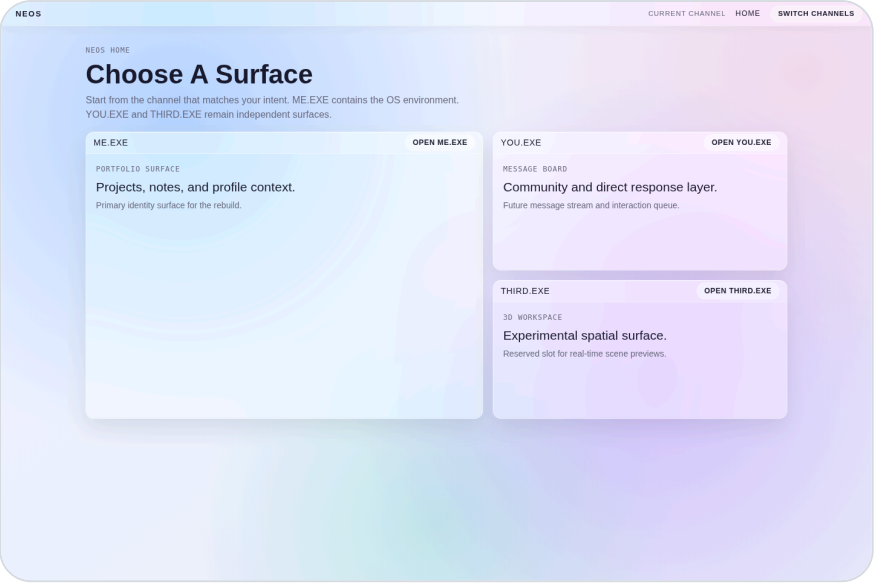
STAGE 1A — ORIGINAL

First Shell UI Pass

This milestone strengthened the HOME landing composition while preserving the scoped ME.EXE OS boundary.

HOME presented a hero-led channel surface layout, and channel switching moved to a secondary quick-switch overlay. ME.EXE owned taskbar and launcher locally.

In plain English: landing felt deliberate and substantial without collapsing into a tile-browser or smart-TV style menu.



Stage 1A: HOME channel navigation surface with hero composition and secondary channel switcher. ME.EXE owned the taskbar in its own channel.

IMPORTANT CODE — STAGE 1A

HOME Composition + Secondary Switcher

```
export function SiteShell(): ReactElement {
  return (
    <main className="neosShell">
      <div className="wallpaper" aria-hidden="true" />
      <ChannelBar ... onOpenSwitcher={() => setSwitcherOpen(true)} />
      <div className="channelViewport">
        <ChannelView channel={activeChannel} onNavigate=
{setActiveChannel} />
      </div>
      {switcherOpen ? <ChannelSwitcher ... /> : null}
      {bootVisible ? <BootOverlay /> : null}
    </main>
  );
}
```

This kept channel switching available but secondary, while preserving ME.EXE as the contained OS environment.

SHELL-01A — V2 REWRITE

Desktop Shell Frame + Boot Gate + Static Taskbar

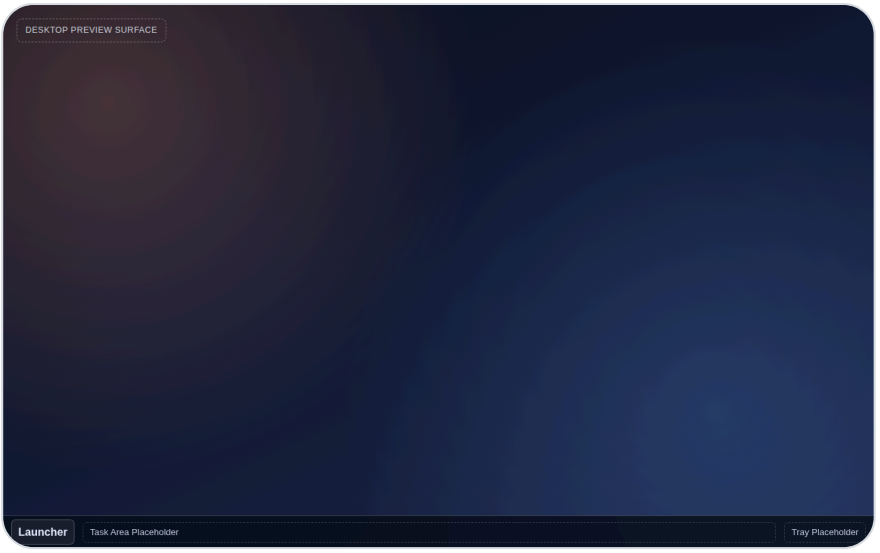
After the v2 workflow reset, the project moved to a contract-first desktop OS rewrite. SHELL-01A is the first milestone of the new architecture: a proper desktop shell frame with correct layer ordering, a boot gate, and a static taskbar.

This is a deliberate reset — not a visual upgrade of Stage 1A, but a structural rewrite with a new architecture. The channel navigation from Stage 1A is preserved in the timeline as history.

What this established

- A stable desktop root in `Desktop.tsx` with spec-correct layer order.
- A contract-shaped desktop store implementing the full `DesktopStore` interface.
- A dedicated `WindowLayer` mount surface with no runtime logic.
- A non-blocking boot gate that does not wait on VFS readiness.
- `Z_LAYER` and `SHELL_Z_INDEX` constants locked in code.

Boot sequence: a fullscreen terminal-style overlay that dismisses on click or keypress. Fires immediately on load without blocking VFS or app services.



Desktop shell after boot is dismissed: gradient wallpaper, a "Desktop Preview Surface" label marking the future icon layer, and a static taskbar with Launcher, Task Area, and Tray placeholder zones.

IMPORTANT CODE — SHELL-01A

Desktop Layer Stack

```
export function Desktop(): JSX.Element {
  const bootComplete = useDesktopStore((store) => store.bootComplete);

  return (
    <main aria-label="Desktop shell" style={desktopStyle}>
      <Wallpaper />
      <div aria-hidden="true" style={previewSurfaceStyle}>
        <span style={previewBadgeStyle}>Desktop Preview Surface</span>
      </div>
      <WindowLayer />
      <Taskbar />
      {!bootComplete ? <BootSequence /> : null}
    </main>
  );
}
```

This stack defines the layer order in code: wallpaper at the base, preview surface, window mount point, taskbar above, and boot overlay on top. Every z-index is driven by named constants, not magic numbers.

What Was Intentionally Deferred in SHELL-01A

- No launcher behavior in this milestone.
- No notification or context-menu systems yet.
- No desktop icon interactions yet.
- No live app windows inside `WindowLayer` yet.
- No CONNECT gameplay or runtime implementation.

Next Milestone Focus

Stage 1B should wire runtime bootstrap and core services without breaking the shell boundaries established in SHELL-01A.

- Add service scaffolding and typed contracts.
- Keep boot non-blocking for VFS readiness.
- Begin window manager integration through the placeholder `WindowLayer`.
- Maintain the desktop as a lightweight preview surface.